RED & WHITE
SKILL EDUCATION
SHAPING SKILLS FOR SCALING HIGHER...!!!

Title: Risk Alert Classifier
Duration: 6 Hours

🎯 Objective

The objective of this project is to design, evaluate, and optimize a classification system that predicts high-risk customer behavior. Students will implement multiple classification algorithms, evaluate models using advanced classification metrics, handle imbalanced data, and improve performance using sampling techniques and hyperparameter tuning.

📑 Problem Statement

You are working as a **Data Scientist** for a digital banking platform. The bank wants to build an **early-warning system** that identifies **high-risk customers** who are likely to default on payments or engage in fraudulent behavior.

The dataset contains customer demographic, behavioral, and transaction-related features. However, the dataset is **highly imbalanced**, where risky customers form a small minority.

Your task is to build a **robust classification pipeline** that:

- Accurately identifies high-risk customers
- Handles class imbalance effectively
- Evaluates performance using appropriate classification metrics
- Improves model accuracy using **hyperparameter tuning**

🧠 Part A: Conceptual Understanding (Theory)

Answer briefly (2–4 lines each):

1. What is **Logistic Regression** and why is it suitable for classification?
2. Explain **classification performance metrics** and why accuracy alone is insufficient.
3. Define **Type-I Error** and **Type-II Error** in the context of risk prediction.
4. Explain **Precision, Recall, F1-Score, TPR, and FPR**.
5. What is **AUC-ROC** and how does it help in evaluating classifiers?
6. Why does **imbalanced data** create problems in classification models?

🧪 Part B: Dataset Understanding & Preparation

You are provided with a dataset containing:

- Customer demographic information
- Transaction activity indicators
- Credit behavior indicators
- Target variable: **Risk Status**
  - 0 → Low Risk
  - 1 → High Risk

Tasks:

7. Identify input features and target variable.
8. Perform a **train–test split** while maintaining class distribution.
9. Identify missing values and apply **KNN Imputer** for multivariate imputation.

📊 Part C: Baseline Classification Model

10. Implement **Logistic Regression** as a baseline model.
11. Generate and interpret:
    - Confusion Matrix
    - Accuracy Score
    - Precision, Recall, F1-Score
12. Identify **Type-I and Type-II errors** from the confusion matrix.

⚖️ Part D: Handling Imbalanced Data

13. Demonstrate the impact of class imbalance on model performance.
14. Apply the following techniques and retrain the model:
    - **Under-Sampling**
    - **Over-Sampling**
    - **SMOTE**
    - **ADASYN**
15. Compare performance before and after balancing using:
    - Recall for minority class
    - F1-Score
    - AUC-ROC

🌳 Part E: Tree-Based Classification Models

16. Implement **Decision Tree Classifier**.
17. Analyze overfitting by comparing training and testing performance.
18. Implement **Random Forest Classifier**.
19. Compare Decision Tree vs Random Forest in terms of accuracy and generalization.

🔧 Part F: Hyperparameter Tuning

20. Apply **Randomized Search CV** to optimize:
    - Decision Tree hyperparameters
    - Random Forest hyperparameters
21. Apply **Grid Search CV** for fine-tuning the best performing model.
22. Compare tuned vs untuned model performance.

📈 Part G: Model Evaluation & ROC Analysis

23. Plot and interpret the **ROC Curve** for all models.
24. Compute and compare **AUC-ROC scores**.
25. Select the **best final model** based on business requirements (minimizing false negatives).

📄 Part H: Final Analysis & Reporting

26. Prepare a final report including:
    - Best classification model and justification
    - Impact of imbalance handling techniques
    - Comparison of performance metrics
    - Business interpretation of false positives and false negatives
27. Submit:
    - Source code / Jupyter Notebook
    - Evaluation tables and plots
    - Final conclusions and recommendations